PHASE 1 – REQUIREMENT ANALYSIS

AI FAQ ASSISTANT API

AI-Powered FAQ and Customer Support Automation

---

1. PROJECT INTRODUCTION

The **AI FAQ Assistant API** is a RESTful backend application developed to simplify FAQ management and provide AI-assisted question answering.

The system allows users to register, log in securely, create and manage FAQs, search existing FAQ information, and use Google Gemini AI to generate answers and FAQ content.

The application is developed using **Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google Gemini AI**. The project follows an **MVC (Model-View-Controller)** architecture.

The main purpose of the system is to provide a simple and secure API that can be used by a future web or mobile application for FAQ management and AI-powered customer support.

Team Members

S.No	Name	Role
1	BALRAJ I	Team Lead
2	BALAKESAVAN S	Team Member
3	BALAMURUGAN E	Team Member
4	BHARATH H	Team Member
5	BHUVANESWARAN G	Team Member

Project Type

Item	Details
Project Name	AI FAQ Assistant API
Application Type	RESTful Backend API
Architecture	MVC
Backend	Node.js + Express.js

Item	Details
Database	MongoDB
Database ODM	Mongoose
Authentication	JWT
Password Security	bcrypt
AI Service	Google Gemini
API Testing	Postman

---

2. PROBLEM STATEMENT

Frequently Asked Questions are commonly used to provide quick answers to repeated questions. However, maintaining FAQ information manually can become difficult when the number of questions increases.

Users may also spend time searching through multiple FAQ records to find the required information. Support personnel may have to repeatedly provide answers to similar questions.

The main problems identified are:

- Manual creation and maintenance of FAQ information.
- Difficulty searching through FAQ records.
- Repeated answering of similar questions.
- Need for secure user authentication.
- Need to restrict FAQ modification to authorized users.
- Need for a centralized FAQ management API.
- Need for AI-assisted answers.
- Need for automatic FAQ generation.
- Need for input validation.
- Need for proper error handling.

The proposed system addresses these problems by combining FAQ management functionality with Google Gemini AI services in a single RESTful backend application.

---

3. PROPOSED SYSTEM

The proposed **AI FAQ Assistant API** provides a centralized backend system for managing frequently asked questions and generating AI-assisted responses.

The system provides three major areas of functionality:

User Authentication

Users can:

- Register an account.
- Log in using their credentials.
- Receive a JWT authentication token.
- Access their profile through a protected endpoint.

FAQ Management

Authenticated users can:

- Create FAQs.
- View FAQs.
- Update their own FAQs.
- Delete their own FAQs.
- Search FAQs.

Public users can view and search FAQ information.

AI Services

Authenticated users can:

- Ask questions and receive AI-generated answers.
- Provide a topic and generate an FAQ question-answer pair using Google Gemini.

The generated FAQ content can subsequently be stored as an FAQ through the FAQ creation API.

---

4. PROJECT OBJECTIVES

The main objectives of the project are:

1. To develop a RESTful API for FAQ management.
2. To provide secure user registration and login.
3. To protect passwords using bcrypt hashing.
4. To implement JWT-based authentication.
5. To allow authenticated users to create FAQs.
6. To allow FAQ creators to update and delete their own FAQs.
7. To provide public FAQ viewing and searching.
8. To provide keyword-based FAQ search.
9. To integrate Google Gemini AI.
10. To generate AI-based answers to user questions.
11. To automatically generate FAQ question-answer pairs.
12. To validate incoming API requests.
13. To provide centralized error handling.
14. To maintain a modular backend architecture.
15. To provide a backend that can later be connected to a web or mobile frontend.

---

5. SYSTEM SCOPE

The current project focuses on the development of the **backend REST API**.

Features Included in the Current System

S.No	Feature	Description
1	User Registration	Creates a new user account
2	User Login	Authenticates registered users
3	JWT Authentication	Protects private endpoints
4	User Profile	Retrieves logged-in user details
5	FAQ Creation	Creates a new FAQ

S.No	Feature	Description
6	FAQ Retrieval	Retrieves FAQ information
7	FAQ Update	Updates an FAQ created by the user
8	FAQ Delete	Deletes an FAQ created by the user
9	FAQ Search	Searches FAQ information
10	AI Answer	Generates answers using Gemini
11	AI FAQ Generation	Generates FAQ question-answer pairs
12	Validation	Validates incoming request data
13	Error Handling	Provides centralized API error handling

Current Scope

The application currently concentrates on the backend and API functionality. The project can later be connected to a frontend application.

Future Scope

Possible future additions include:

- React frontend
- Admin interface
- Advanced user roles
- Search history
- Notifications
- Analytics
- Advanced semantic/vector search
- Cloud deployment

---

6. FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system should perform.

6.1 User Registration

The system shall allow a new user to register by providing:

- Name
- Email
- Password

The system shall check whether the email already exists before creating a new account.

6.2 User Login

The system shall allow registered users to log in using:

- Email
- Password

After successful authentication, the system shall generate and return a JWT token.

6.3 User Profile

The system shall provide a protected profile endpoint.

Only an authenticated user with a valid JWT token shall be able to access the profile information.

6.4 FAQ Creation

Authenticated users shall be able to create FAQ records.

Each FAQ shall contain:

- Question
- Answer
- Category
- Creator information

The creator's user ID shall be associated with the FAQ.

6.5 FAQ Retrieval

The system shall allow users to:

- View all FAQs.
- View an individual FAQ using its ID.

These retrieval operations are available without authentication in the current implementation.

6.6 FAQ Update

An authenticated user shall be able to update an FAQ created by that user.

The system shall verify ownership before allowing the update operation.

6.7 FAQ Delete

An authenticated user shall be able to delete an FAQ created by that user.

The system shall verify the creator before deleting the FAQ.

6.8 FAQ Search

The system shall provide a search endpoint using a query parameter.

The search operation checks FAQ:

- Questions
- Answers
- Categories

The current implementation uses MongoDB regular-expression matching for keyword-based searching.

6.9 AI Answer Generation

Authenticated users shall be able to submit a question to the AI Answer endpoint.

The question is sent to Google Gemini and the generated answer is returned through the API.

6.10 AI FAQ Generation

Authenticated users shall be able to provide a topic.

The AI service generates:

- FAQ question
- FAQ answer

The result is returned in structured JSON format.

Functional Requirement Summary

ID	Requirement	Description
FR01	Registration	Register a new user
FR02	Login	Authenticate a user
FR03	Profile	Retrieve user profile
FR04	Create FAQ	Add FAQ information
FR05	Read FAQ	View FAQ information
FR06	Update FAQ	Modify own FAQ
FR07	Delete FAQ	Delete own FAQ
FR08	Search FAQ	Search FAQ records
FR09	AI Answer	Generate AI answer
FR10	AI FAQ	Generate FAQ using AI

---

7. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality and operational characteristics of the system.

Table 7.1 – Non-Functional Requirements

Requirement	Description
Security	Protect private API operations using authentication
Performance	Process normal API requests efficiently
Reliability	Handle errors without terminating the application unexpectedly
Maintainability	Keep controllers, routes, models and services separated
Scalability	Allow additional features to be added later
Data Integrity	Validate data before database storage
Usability	Provide simple and consistent API responses
Compatibility	Support modern Node.js environments
Availability	API should remain accessible while the server is running

Security Requirements

The system shall provide:

- JWT token verification.
- Password hashing using bcrypt.
- Protected routes.
- Owner verification for FAQ modification.
- Input validation.
- Centralized error handling.
- Environment-based configuration for sensitive values.

---

8. TECHNOLOGY AND SYSTEM REQUIREMENTS

8.1 Software Requirements

Technology	Purpose
Node.js	Backend runtime environment
Express.js	REST API development
MongoDB	Database
Mongoose	MongoDB object modelling
JSON Web Token	User authentication
bcrypt	Password hashing
Google Gemini	AI functionality
dotenv	Environment variable management
CORS	Cross-origin request handling
Postman	API testing
Visual Studio Code	Development environment

The application requires **Node.js 18+ or 20+**, MongoDB or MongoDB Atlas, and a Google Gemini API key.

8.2 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i5 / Equivalent
RAM	8 GB or 16 GB
Storage	At least 1 GB free space
Internet	Required for Gemini API
Display	Standard display
Input	Keyboard and mouse

8.3 Environment Configuration

The application uses environment variables for important configuration values.

The main configuration values are:

Variable	Purpose
PORT	Defines the server port
MONGO_URI	MongoDB connection string
JWT_SECRET	Secret used for JWT authentication
GEMINI_API_KEY	Google Gemini API key

Sensitive values should not be directly exposed in the application source code.

---

9. SYSTEM MODULES AND BASIC ARCHITECTURE

The system is divided into the following modules.

9.1 Authentication Module

Responsible for:

- Registration
- Login
- Password hashing
- JWT generation
- Profile access

9.2 FAQ Module

Responsible for:

- FAQ creation
- FAQ retrieval
- FAQ update
- FAQ deletion
- FAQ search

9.3 AI Module

Responsible for:

- AI-generated answers
- AI-generated FAQ question-answer pairs

9.4 Database Module

Responsible for:

- MongoDB connection
- User data storage
- FAQ data storage
- Data validation

9.5 Middleware Module

Responsible for:

- JWT verification
- Request validation
- Error handling

9.6 Service Module

The Gemini service handles communication between the application and Google Gemini AI.

Basic Architecture

The application follows an MVC-based backend structure:

Client / Postman

↓

Express.js Application

↓

Routes

↓

Middleware

↓

Controllers

↓

Models / Services

↓

MongoDB / Google Gemini

Project Structure

Folder/File	Purpose
config/	Database configuration
controllers/	Application request processing
middleware/	Authentication, validation and errors
models/	MongoDB schemas
routes/	API route definitions
services/	Gemini AI integration
utils/	Helper functions
app.js	Express application configuration
server.js	Server startup and database connection

Phase 1 should contain only one simple architecture diagram here, if required. Do not add screenshots of the code yet.

---

10. REQUIREMENT SUMMARY AND PHASE PLAN

10.1 Database Requirements

The current system uses MongoDB with two main models.

User

Field	Purpose
Name	Stores user's name
Email	Stores unique user email
Password	Stores hashed password
Created At	Records creation time
Updated At	Records modification time

FAQ

Field	Purpose
Question	Stores FAQ question
Answer	Stores FAQ answer
Category	Stores FAQ category
Created By	Identifies FAQ creator
Created At	Records creation time
Updated At	Records modification time

The relationship between the two models is:

One User → Many FAQs

Each FAQ contains a reference to the user who created it.

10.2 Main API Requirements

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register user
POST	/api/auth/login	Public	Login user
GET	/api/auth/profile	Protected	Get profile
POST	/api/faqs	Protected	Create FAQ
GET	/api/faqs	Public	Get FAQs
GET	/api/faqs/:id	Public	Get specific FAQ
PUT	/api/faqs/:id	Protected	Update own FAQ
DELETE	/api/faqs/:id	Protected	Delete own FAQ
GET	/api/faqs/search?q=	Public	Search FAQs
POST	/api/ai/answer	Protected	Generate AI answer
POST	/api/ai/generate-faq	Protected	Generate FAQ using AI

10.3 Expected Outcome

At the end of the project, the system is expected to provide:

- Secure user authentication.
- JWT-based protected API access.
- FAQ creation and management.
- FAQ searching.
- Owner-based FAQ modification.
- Google Gemini AI answer generation.
- Google Gemini AI FAQ generation.
- MongoDB data storage.
- Input validation.
- Centralized error handling.
- API testing through Postman.

10.4 Phase-Wise Development Plan

Phase	Main Work
Phase 1	Requirement Analysis
Phase 2	System Design and Architecture
Phase 3	Project Setup and Backend Development
Phase 4	Authentication and Security
Phase 5	FAQ Management
Phase 6	Google Gemini AI Integration
Phase 7	API Testing and Validation
Phase 8	Final Integration and Results

Screenshot Planning

To keep the documentation clean, screenshots will be introduced mainly in the later phases.

Phase	Suggested Screenshots
Phase 1	Architecture diagram only, if required
Phase 2	Architecture/database diagrams
Phase 3	VS Code project structure and server
Phase 4	Registration, login and JWT results
Phase 5	FAQ CRUD and search results
Phase 6	Gemini AI answer and FAQ generation
Phase 7	Postman testing and test results
Phase 8	Final complete project output

CONCLUSION

The **AI FAQ Assistant API** is a backend application designed to provide secure FAQ management and AI-powered assistance.

The system combines **Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google Gemini AI** to provide authentication, FAQ management, keyword-based search, AI-generated answers, and automatic FAQ generation.

The requirement analysis establishes the functional and non-functional requirements of the system and defines the technology, database, API, security, and module requirements.

The project will be developed phase by phase, with implementation screenshots and testing evidence introduced in the appropriate later phases. This keeps the Phase 1 document focused on **requirements rather than implementation screenshots** and prevents the same screenshots from being repeated throughout the project documentation.